MinutiaeNet matching test: with vs. without preprocessing

Setup

- **Enrollment**: `image_db/Rahul.jpg` (1 photo)
- **Probes**: all 106 photos in `test_db/Rahul/`
- **Matcher**: `backend/matching.py` (`match_templates`, "modern" algorithm — rotation-invariant kNN graph matching with relaxation labeling)
- **Models**: `single_finger_backend_models/best-new.pt` (YOLO finger detector), `best_f1.pth` (MinutiaeNet). With-preprocessing also uses `u2net_320x320_float32.tflite` and `zero_dce_model.h5`.
- Same single identity throughout (Rahul), so this measures **genuine-match accuracy vs. threshold** only — not FAR or EER, which need impostor pairs from other identities and were explicitly out of scope for this run.
- Source photos are raw 8192x6144 (50MP) camera captures. Both scripts downscale to a 1600px long edge before running the pipeline (common.py:`MAX_DIM`) — this is a test-harness change only, `backend/app.py` itself is untouched.

Without preprocessing (without-preprocessing/run_without_preprocessing.py)

YOLO crop → MinutiaeNet directly. No U²-Net segmentation, no Zero-DCE/histogram-equalization enhancement.

With preprocessing (with-preprocessing/run_with_preprocessing.py)

YOLO crop → U²-Net segmentation mask → `preprocess_fingerprint` (Zero-DCE if luminance < 150, else histogram equalization) → MinutiaeNet. Uses `backend/app.py`'s real `load_models()` / `get_segmentation_mask` / `preprocess_fingerprint` / `detect_minutiae` unmodified.

Both runs: **0 extraction failures** out of 106 probes.

Accuracy by threshold

Threshold	Without preprocessing	With preprocessing
0.1	100.00% (106/106)	100.00% (106/106)
0.2	100.00% (106/106)	99.06% (105/106)
0.3	66.98% (71/106)	**91.51% (97/106)**
0.4	20.75% (22/106)	**35.85% (38/106)**
0.5	4.72% (5/106)	4.72% (5/106)
0.6	0.94% (1/106)	0.94% (1/106)
0.7	0.94% (1/106)	0.94% (1/106)
0.8	0.94% (1/106)	0.94% (1/106)
0.9	0.94% (1/106)	0.94% (1/106)

Full per-threshold data: without-preprocessing/results_summary.csv, with-preprocessing/results_summary.csv. Per-image scores: `results_per_image.csv` in each folder.

Score distribution

	Without preprocessing	With preprocessing
Mean score	0.3495	0.3859
Median score	0.3446	0.3768
Std dev	0.0971	0.0886
Min / Max	0.2036 / 0.9747	0.1998 / 0.9866
Avg. minutiae/image	69.6	55.7

Reading the numbers

- **0.1–0.2 is not a usable operating threshold** — accuracy is saturated near 100% for both because the threshold is too lenient to be discriminative (nothing has been rejected yet to reveal a difference).
- **The real signal is at 0.3–0.4**: preprocessing raises accuracy from 66.98% → 91.51% at 0.3, and from 20.75% → 35.85% at 0.4. Both distributions have similar means, but preprocessing gives a tighter, less variable score (lower std dev, higher median), which is what actually improves how many probes clear a stricter threshold.
- Preprocessing also produces noticeably fewer minutiae per image on average (55.7 vs 69.6) — consistent with the segmentation mask removing background/edge points and enhancement suppressing spurious low-contrast detections, leaving a cleaner but smaller point set.
- Above 0.5, both pipelines collapse to essentially one image (`17.jpg`, a near-duplicate/very-similar capture scoring 0.97–0.99 in both runs) — at that threshold neither pipeline is usable for real authentication with this capture set.
- **Bottom line for this identity's data**: preprocessing helps most in the 0.3–0.4 threshold band, where it needs to matter — the region where a system would realistically set its accept/reject cutoff. It doesn't change what threshold gives 100% (0.1–0.2), but it substantially widens the margin before accuracy collapses as the threshold tightens.

Caveats

- Single identity (Rahul) — this is genuine-match accuracy only, not a security metric. No impostor data yet, so FAR/EER can't be computed from this run alone.
- One enrollment photo; no cross-photo enrollment averaging.
- Not a speed benchmark — with-preprocessing added ~2 CPU-seconds/image over without-preprocessing (mostly U²-Net + Zero-DCE), on downscaled 1600px images. The CLAUDE.md 5-second on-device budget assumes on-device execution, which isn't wired up yet in either pipeline.